Universidad Católica Boliviana



INFORME TÉCNICO

Herbario de Flora Boliviana

Arquitectura orientada a servicios con API Gateway, microservicio NestJS y PostgreSQL, orquestada con Docker Compose

Estudiante	Luz Camila Pérez Cano
Docente	Tecnologías Web 2
Tecnología asignada	NestJS (TypeScript)
Base de datos	PostgreSQL
Repositorio	[URL del repositorio en GitHub]
Fecha	Octubre de 2026

1. Descripción del dominio

1.1 Problema de negocio

Bolivia es un país con una diversidad vegetal enorme, repartida en ecorregiones muy distintas: el Altiplano, los valles, los Yungas, el Chaco, la Amazonía y los Llanos. Aun así, la información sobre su flora suele estar dispersa en planillas, cuadernos de campo e informes aislados. Por eso es difícil saber qué especies existen, a qué familia botánica pertenecen, a qué altitud viven, en qué estado de conservación se encuentran y, sobre todo, dónde y cuándo fue observada cada una. No existe una fuente única que se pueda consultar ni un registro que permita rastrear cada dato.

1.2 Solución propuesta

Se desarrolló un herbario digital expuesto como API REST. El sistema guarda el catálogo de plantas junto con su familia, ecorregión, rango de altitud y estado de conservación, y permite registrar avistamientos con localidad, departamento, coordenadas, fecha y observador. Cualquier cliente (una aplicación web o móvil, un script de investigación) accede por un solo punto de entrada, el API Gateway, que autentica y controla el tráfico antes de que la petición llegue al microservicio.

1.3 Actores

Actor	Papel en el sistema
Investigador botánico	Registra y actualiza plantas, familias y avistamientos de campo.
Cliente de la API	Aplicación o persona que consulta el catálogo mediante búsquedas y filtros.
Administrador	Tiene las credenciales del gateway y gestiona la configuración del despliegue.

1.4 Entidades

Entidad	Atributos principales	Relaciones
Familia	id, nombre (único), descripción	Una familia agrupa muchas plantas (1:N).
Planta	id, nombre científico (único), nombre común, descripción, ecorregión, altitud mínima y máxima, estado de conservación	Pertenece a una familia (N:1) y tiene muchos avistamientos (1:N).
Avistamiento	id, localidad, departamento, latitud, longitud, fecha, observador, notas	Pertenece a una planta (N:1).

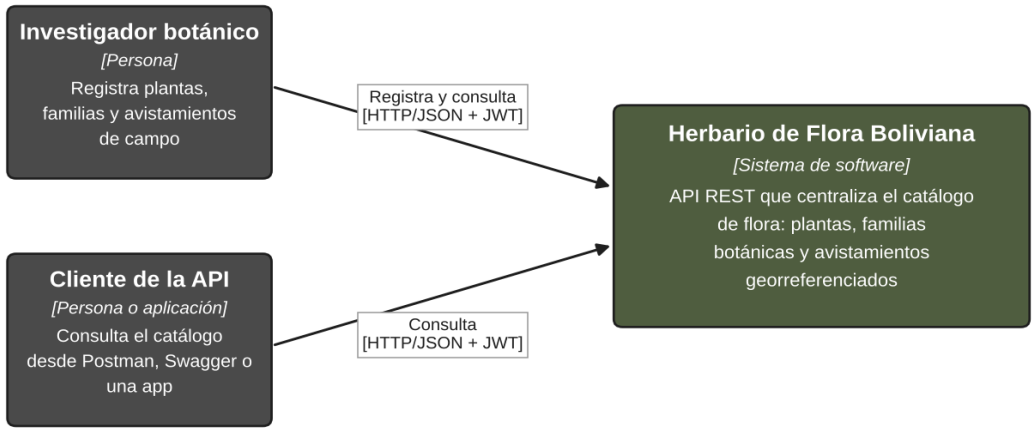
1.5 Reglas de negocio

- El nombre científico de la planta y el nombre de la familia son únicos; si se repiten, el sistema responde con un conflicto (409).
- Toda planta debe pertenecer a una familia que ya exista.
- No se puede eliminar una familia que todavía tiene plantas asociadas (restricción RESTRICT, respuesta 409). Al eliminar una planta se eliminan también sus avistamientos (CASCADE).
- La altitud mínima no puede ser mayor que la máxima (error 400).
- El estado de conservación solo admite cinco categorías, tomadas de la Lista Roja de la UICN: preocupación menor, casi amenazada, vulnerable, en peligro y en peligro crítico.

- El departamento del avistamiento debe ser uno de los nueve departamentos de Bolivia, y la fecha se escribe en formato AAAA-MM-DD.

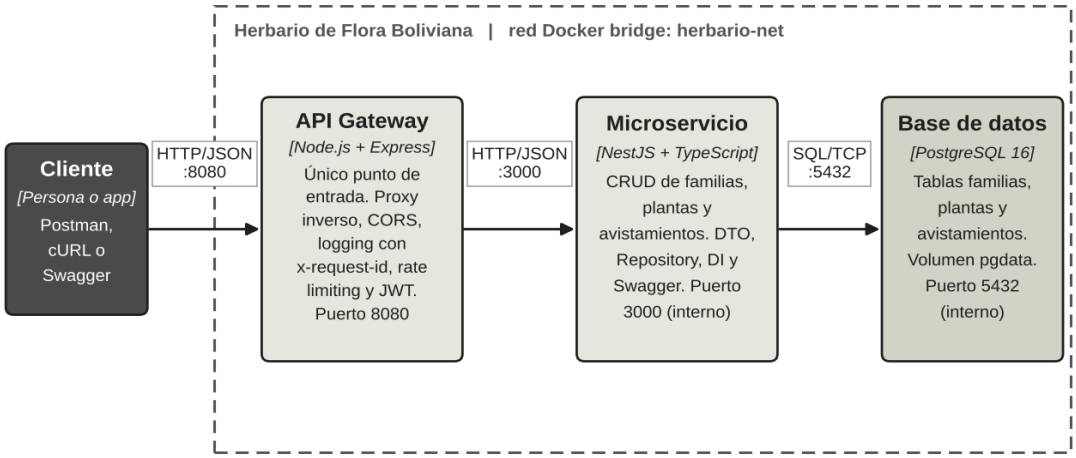
2. Arquitectura del sistema (modelo C4)

2.1 Nivel 1: diagrama de contexto



El sistema es consumido por dos tipos de actores, personas o aplicaciones, siempre mediante HTTP/JSON y con un token JWT. No depende de sistemas externos: toda la información vive en su propia base de datos.

2.2 Nivel 2: diagrama de contenedores



Contenedor	Tecnología	Responsabilidad	Exposición
API Gateway	Node.js 20, Express, http-proxy-middleware	Punto único de entrada. Enruta /api y /docs; aplica CORS, logging con x-request-id, rate limiting y JWT.	Puerto 8080 publicado en el host
Microservicio	NestJS 10, TypeScript, TypeORM, class-validator, Swagger	Lógica de negocio y CRUD de familias, plantas y avistamientos (DTO, Repository, DI).	Puerto 3000, solo red interna

Contenedor	Tecnología	Responsabilidad	Exposición
Base de datos	PostgreSQL 16 (imagen alpine)	Persistencia relacional con el volumen pgdata.	Puerto 5432, solo red interna

2.3 Decisiones de diseño

Gateway como única puerta. Solo el puerto 8080 se publica hacia el host. El microservicio y PostgreSQL quedan dentro de la red herbario-net (driver bridge) y no se pueden alcanzar desde fuera.

Características transversales. El gateway aplica CORS configurable, registra cada petición en formato JSON con un identificador x-request-id que viaja hasta el microservicio, limita el tráfico (100 peticiones por minuto por IP y 10 intentos por minuto en el login) y valida el JWT en todas las rutas /api.

Patrones en el microservicio. Los servicios inyectan Repository<Entidad> de TypeORM (patrón Repository), las entradas se validan con DTO (whitelist y forbidNonWhitelisted) y todo se conecta con la inyección de dependencias de NestJS.

Contenerización. El microservicio usa un Dockerfile multi-stage: compila en una etapa y copia solo lo necesario a la imagen final, que corre con un usuario no root. El gateway usa un Dockerfile liviano. En docker-compose.yml los tres servicios tienen healthcheck y depends_on con la condición service_healthy, de modo que primero arranca la base de datos, luego el microservicio y al final el gateway.

Manejo de errores. Las violaciones de unicidad y de llaves foráneas de PostgreSQL se convierten en respuestas 409. Si el microservicio deja de responder, el gateway devuelve 502 en lugar de quedarse esperando.

3. Documentación de endpoints

Todas las rutas se consumen a través del gateway, en http://localhost:8080. Las rutas /api/* exigen el encabezado Authorization: Bearer <token>. La documentación interactiva (Swagger UI) está en /docs.

Método	URL	Descripción	Respuestas
POST	/auth/login	Autentica y devuelve el JWT	200, 401, 429
GET	/health	Estado del gateway	200
GET	/api/familias	Lista las familias	200
GET	/api/familias/:id	Obtiene una familia	200, 404
POST	/api/familias	Crea una familia	201, 400, 409
PATCH	/api/familias/:id	Actualiza una familia	200, 400, 404, 409
DELETE	/api/familias/:id	Elimina una familia (sin plantas)	204, 404, 409
GET	/api/plantas	Lista plantas; filtros familiaIdj, ecorregion, estado y q	200, 400
GET	/api/plantas/:id	Obtiene una planta con su familia	200, 404
GET	/api/plantas/:id/avistamientos	Avistamientos de una planta	200, 404
POST	/api/plantas	Crea una planta	201, 400, 409
PATCH	/api/plantas/:id	Actualiza una planta	200, 400, 404, 409

Método	URL	Descripción	Respuestas
DELETE	/api/plantas/:id	Elimina una planta y sus avistamientos	204, 404
GET	/api/avistamientos	Lista avistamientos; filtros plantaId y departamento	200, 400
GET	/api/avistamientos/:id	Obtiene un avistamiento	200, 404
POST	/api/avistamientos	Registra un avistamiento	201, 400, 409
PATCH	/api/avistamientos/:id	Actualiza un avistamiento	200, 400, 404, 409
DELETE	/api/avistamientos/:id	Elimina un avistamiento	204, 404

Si falta el token o es inválido, el gateway responde 401; si se excede el límite de peticiones, 429; y si el microservicio no responde, 502.

3.1 Estructuras JSON

Login: POST /auth/login

Solicitud: {"username": "admin", "password": "herbario123"}

Respuesta 200: {"access_token": "<jwt>", "token_type": "Bearer", "expires_in": "1h"}

Crear familia: POST /api/familias

Solicitud: {"nombre": "Cactaceae", "descripcion": "Familia de los cactus"}

Respuesta 201: {"id": 6, "nombre": "Cactaceae", "descripcion": "Familia de los cactus"}

Crear planta: POST /api/plantas

Solicitud:

{ "nombreCientifico": "Echinopsis tarijensis", "nombreComun": "Cardón",
"ecorregion": "Valles secos", "altitudMin": 1800, "altitudMax": 3200,
"estadoConservacion": "preocupacion_menor", "familiaId": 6 }

Respuesta 201:

{ "id": 6, "nombreCientifico": "Echinopsis tarijensis", "nombreComun": "Cardón",
"ecorregion": "Valles secos", "altitudMin": 1800, "altitudMax": 3200,
"estadoConservacion": "preocupacion_menor", "familiaId": 6,
"familia": { "id": 6, "nombre": "Cactaceae", "descripcion": "Familia de los cactus" } }

Registrar avistamiento: POST /api/avistamientos

Solicitud:

{ "plantaId": 6, "localidad": "Tarija - Tolomosa", "departamento": "Tarija",
"latitud": -21.5, "longitud": -64.8, "fecha": "2026-10-10",
"observador": "Estudiante UCB", "notas": "Ejemplar adulto" }

Respuesta 201: el avistamiento creado, con su "id" y la "planta" asociada.

Errores communes

400: {"message": ["nombreCientifico should not be empty"], "error": "Bad Request"}

401: {"error": "Unauthorized", "message": "Falta el token Bearer."}

409: {"message": "Ya existe un registro con esos datos únicos.", "error": "Conflict"}

502: {"error": "Bad Gateway", "message": "El microservicio no responde."}

4. Evidencia de pruebas

Las pruebas se ejecutaron con el script scripts/smoke-test.sh (20 casos de punta a punta contra el gateway) y con la colección de Postman incluida en el repositorio. Las secciones 4.1 a 4.4 muestran la salida de esa ejecución; la sección 4.5 reúne las capturas del sistema corriendo con Docker Compose, Swagger y Postman.

4.1 Resultados de la batería de pruebas

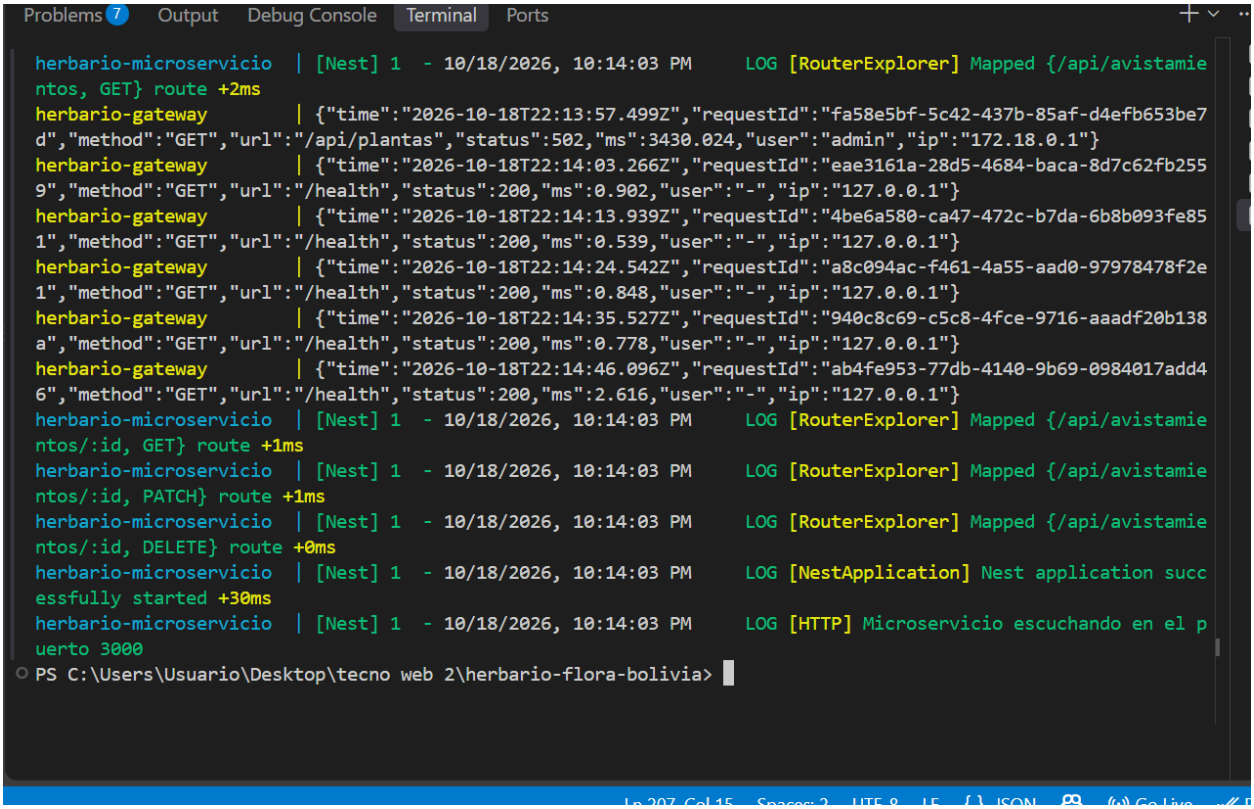
N.º	Caso de prueba	Petición	Esperado	Obtenido
1	Health del gateway	`GET /health`	200	200 OK
2	Acceso sin token	`GET /api/plantas`	401	401 OK
3	Login con credenciales incorrectas	`POST /auth/login`	401	401 OK
4	Login correcto	`POST /auth/login`	200	200 OK
5	Listar familias	`GET /api/familias`	200	200 OK
6	Crear familia	`POST /api/familias`	201	201 OK
7	Crear familia duplicada	`POST /api/familias`	409	409 OK
8	Crear planta	`POST /api/plantas`	201	201 OK
9	Crear planta con datos inválidos	`POST /api/plantas`	400	400 OK
10	Buscar plantas por nombre	`GET /api/plantas?q=cardón`	200	200 OK
11	Filtrar plantas en peligro	`GET /api/plantas?estado=en_peligro`	200	200 OK
12	Obtener planta por id	`GET /api/plantas/8`	200	200 OK
13	Actualizar planta	`PATCH /api/plantas/8`	200	200 OK
14	Registrar avistamiento	`POST /api/avistamientos`	201	201 OK
15	Avistamientos de la planta	`GET /api/plantas/8/avistamientos`	200	200 OK
16	Borrar familia con plantas	`DELETE /api/familias/12`	409	409 OK
17	Borrar avistamiento	`DELETE /api/avistamientos/4`	204	204 OK
18	Borrar planta	`DELETE /api/plantas/8`	204	204 OK
19	Borrar familia	`DELETE /api/familias/12`	204	204 OK
20	Planta borrada ya no existe	`GET /api/plantas/8`	404	404 OK

4.2 Ejemplos de salida

```
HTTP/1.1 401 Unauthorized
x-request-id: 49f5bb33-ac25-4e17-8693-8f5eac349f17
Vary: Origin
Access-Control-Expose-Headers: x-request-id
RateLimit-Policy: 100;w=60
RateLimit-Limit: 100
RateLimit-Remaining: 98
RateLimit-Reset: 56
Content-Type: application/json; charset=utf-8
Content-Length: 59
ETag: W/"3b-1VCNVGcZ91U7KhhGt7rU/t+FcJ0"
Date: Sun, 18 Oct 2026 22:11:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{"error":"Unauthorized","message":"Falta el token Bearer."}
PS C:\Users\Usuario\Desktop\tecno web 2\herbario-flora-bolivia> $t = (Invoke-RestMethod -Method Post -Uri htt
p://localhost:8080/auth/login -ContentType "application/json" -Body '{"username":"admin","password":"herbario
123"}').access_token
>> curl.exe -i -X POST http://localhost:8080/api/familias -H "Authorization: Bearer $t" -H "Content-Type: app
lication/json" -d '{\"nombre\":\"Cactaceae\"}'
HTTP/1.1 409 Conflict
x-request-id: c99429c5-b0cd-44ef-a37a-64a924a40819
Vary: Origin
Access-Control-Expose-Headers: x-request-id
RateLimit-Policy: 100;w=60
RateLimit-Limit: 100
RateLimit-Remaining: 96
```

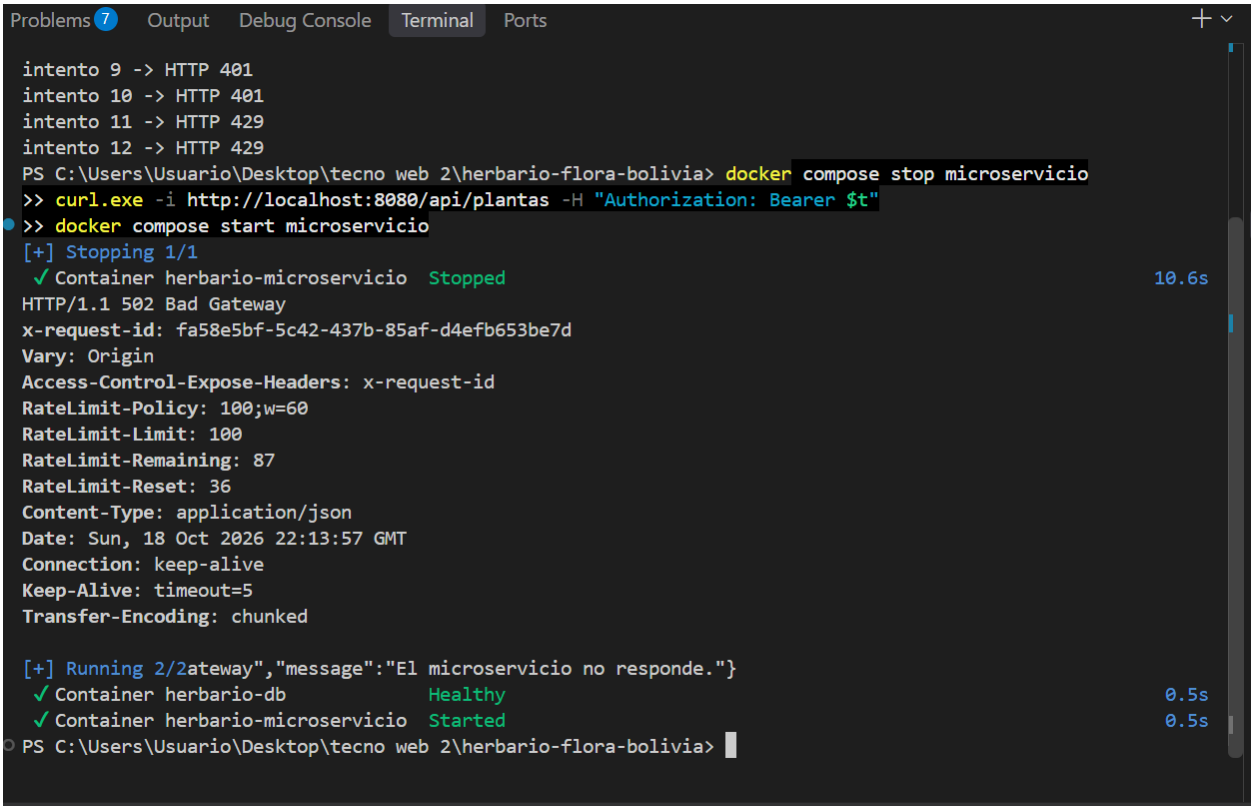
4.3 Trazabilidad con x-request-id

El gateway asigna un identificador a cada petición y lo envía al microservicio. El mismo valor aparece en los logs de ambos, lo que permite seguir una petición de extremo a extremo.



4.4 Rate limiting y error 502

Con diez intentos de login por minuto como máximo, el intento número 11 recibe 429. Al detener el microservicio, el gateway responde 502 y deja el error en su log.



4.5 Capturas de Docker, Swagger

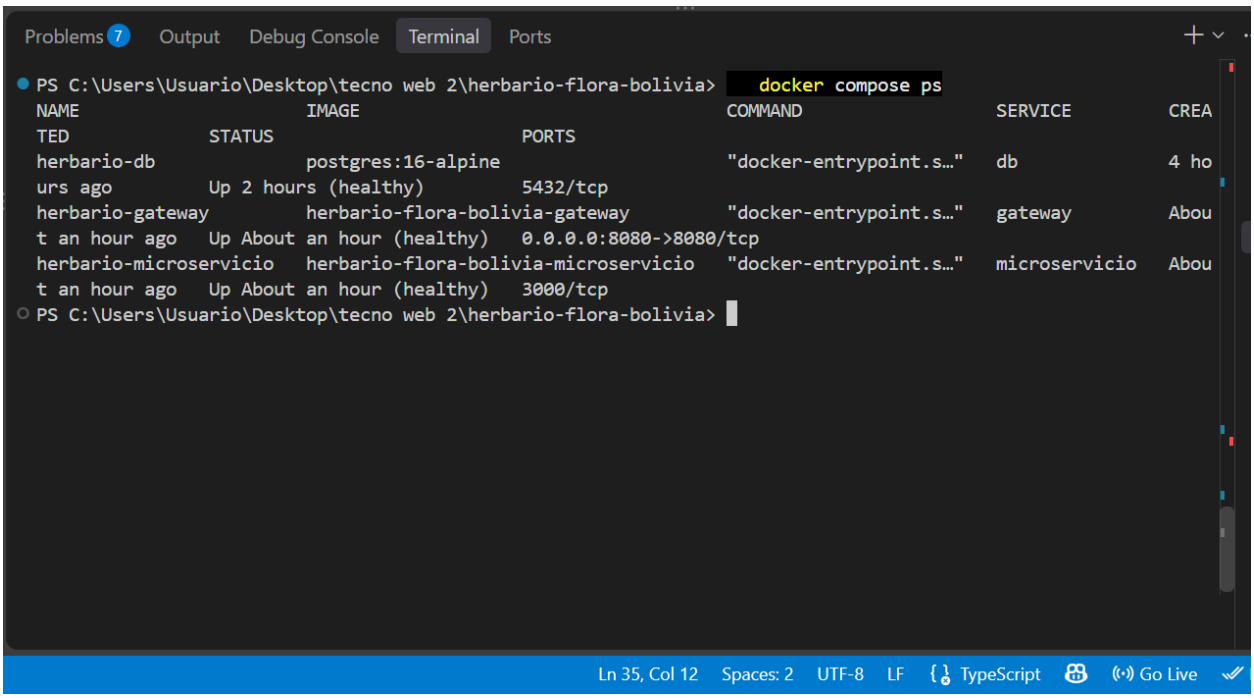


Figura 3. Estado de los contenedores.

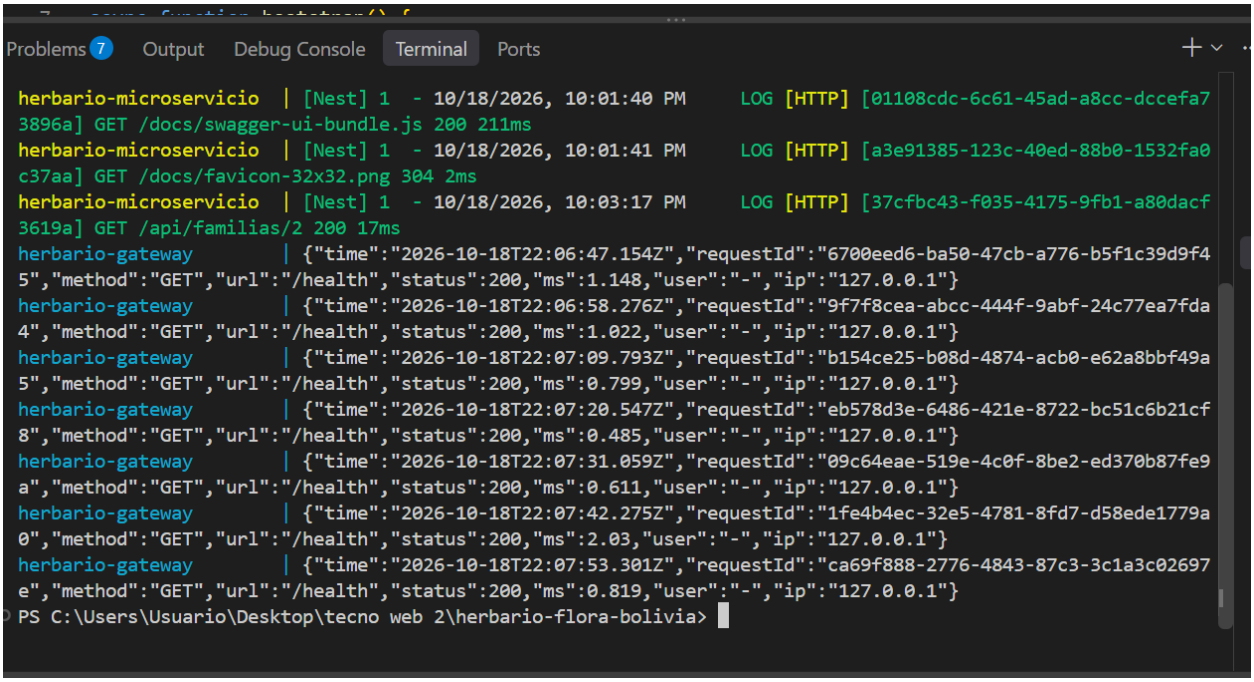


Figura 4. Logs de los contenedores.

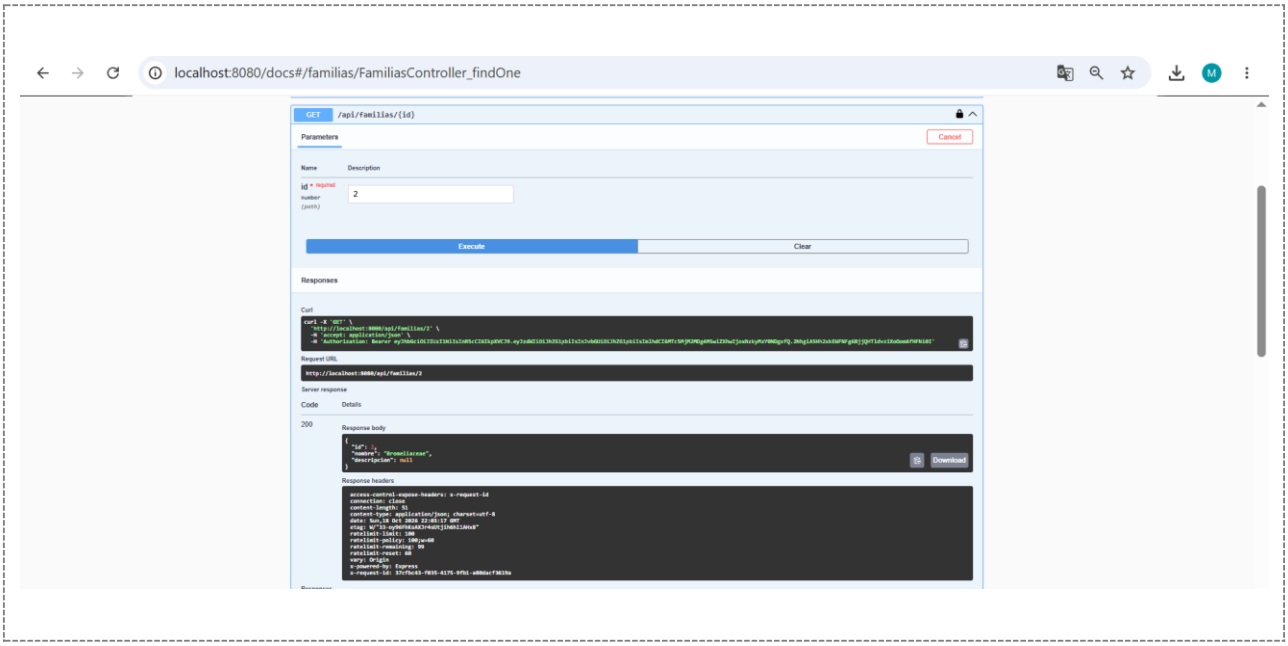


Figura 5. Prueba en Swagger UI.

Conclusión

La arquitectura cumple lo pedido. El gateway centraliza el acceso con CORS, logging con trazabilidad, rate limiting y JWT; el microservicio NestJS ofrece un CRUD completo sobre PostgreSQL con los patrones Repository, DTO e inyección de dependencias; y todo se levanta con un solo comando, docker compose up --build.